

Projektdokumentation

PotensicProxy_TAF - Live-System-Logs und Kamera-/Medien-Diagnosekorrekturen

Dokumentart	Projektdokumentation / Umsetzungsnachweis
Projekt	PotensicProxy_TAF
Datum	2026-10-02
Version	v1.4
Status	Software umgesetzt - reale Geräteabnahme teilweise offen
Verantwortlich	OpenAI / ChatGPT
Änderungsauftrag	Live-System-Logs / Kamera-Diagnose v1.1
Referenzen	Potensic_Atom_2 Protokollreferenz v0.6; Projektdokumentationsregeln
Ablageort	DOCUMENTATION/ im Projektpaket

1. Anlass und Ziel

Der Änderungsauftrag v1.1 verlangte zwei zusammenhängende Korrekturgruppen: eine begrenzte und exportierbare Live-System-Log-Anzeige sowie technische Korrekturen im Kamera-, Galerie- und Downloadpfad. Die Umsetzung wurde auf dem gelieferten Stand PotensicProxy_TAF vorgenommen. Der Auftrag fordert unter anderem einen 1000-Zeilen-Rolling-Buffer, Save Log, korrekte Behandlung von 0x39, readiness-synchronisierten Capture, fragmentierte 0x20-Metadaten, 0x1B-Teilframes und einen definierten Passthrough-Reconnect.

2. Umgesetzte Änderungen

2.1 Live-System-Logs

- Der Live-Log-Puffer in useDroneStore.ts ist von 500 auf verbindlich maximal 1000 Einträge erhöht und als Rolling Buffer umgesetzt.
- LogConsole.vue zeigt permanent den Zähler „n / 1000 lines“ und hält Header, Save Log und Clear außerhalb des scrollenden Log-Inhalts sichtbar.
- Save Log erzeugt clientseitig einen konsistenten Snapshot des aktuell gehaltenen Puffers als UTF-8-Textdatei mit CRLF-Zeilenenden.
- Dateiname: PotensicProxy_TAF_LiveLog_YYYY-MM-DD_HH-mm-ss.log. Der Export verändert den Puffer nicht.

2.2 Camera Log 0x39 und Mode-Notification 0x3A

CameraMediaService.handleIncoming() unterscheidet asynchrone Kamera-Datensätze jetzt vor der generischen Statusauswertung. 0x39 wird als CameraLogData protokolliert und kann dadurch nicht mehr als „Camera command 0x39 failed“ fehlinterpretiert werden. 0x3A wird als asynchrone Mode-Switch-Notification verarbeitet und in die Capture-Bereitschaftslogik einbezogen.

2.3 Capture-State-Machine

- Vor Capture werden Galerie-/Listen-/Metadatenzustände beendet; ein aktiver Datei-Download wird nicht parallel überfahren, sondern blockiert Capture sichtbar.
- Nach 0x03-Mode-ACK wird Capture nicht mehr sofort gesendet. TAF fordert 0x02 an und akzeptiert zusätzlich 0x3A als Bereitschaftssignal.
- Erst wenn der Zielmodus PHOTO bzw. VIDEO bestätigt ist, wird 0x01 bzw. 0x00 01/00 00 gesendet.
- Status 3 „Device busy“ beendet den laufenden ACK-Timer, wartet 350 ms, synchronisiert mit 0x02 und erlaubt höchstens einen kontrollierten Retry.
- Späte 0x03-/0x02-Antworten lösen keinen doppelten Capture aus, wenn bereits ein Foto-/Video-Command pending ist.

2.4 Fragmentierte 0x20-Metadaten

Die Antwort auf FILE_META_LIST wird in einem Response-Accumulator gesammelt. JSON.parse() wird erst aufgerufen, wenn eine vollständige JSON-Struktur erkannt wurde. Unvollständige Fragmente werden weitergepuffert. Der Accumulator wird bei erfolgreichem Abschluss, Timeout, Galerieende und Passthrough-Abbruch verworfen. Eine Sicherheitsgrenze von 1.000.000 Zeichen verhindert unbegrenztes Wachstum.

2.5 Download 0x1B

Download-Daten werden zunächst in einen Stream-Puffer übernommen. Aus flag und dem protokollseitigen u16-Payload-Längenfeld wird die erwartete Framegröße bestimmt; unvollständige Frames werden erst nach vollständigem Eingang ausgewertet. Die Unit-Steuerung folgt nun dem Änderungsauftrag: flag=0 schreibt Teilblock und wartet ohne neuen 0x1B-Request; flag=1 beendet die aktuelle Unit und startet erst dann die nächste Anfrage; flag=2 finalisiert die Datei. Gültiger Datenfortschritt aktualisiert den Inaktivitätszeitpunkt. Duplicate-Frames lösen keine zusätzliche Anfrage aus; Offset-Abweichungen werden vom lokal bestätigten Offset neu angefordert.

2.6 Passthrough-Reconnect

UsbTransportService setzt beim WebSocket-Abbruch den FE-Demuxer zurück und ruft CameraMediaService.handleTransportDisconnect() auf. Pending Capture-, Metadata-, Delete- und Download-Zustände werden beendet. Ein Download wird mit sichtbarem Fehler am bestätigten Offset abgebrochen und nicht still fortgesetzt. Beim Reconnect wird der Kameramodus auf UNKNOWN gesetzt und sofort 0x02 zur Status-Synchronisation gesendet.

3. Geänderte Dateien

Datei	Änderung
webui/src/components/logs/LogConsole.vue	Sticky/ständiger Kopfbereich, Zeilenzähler, Save-Log-Export.
webui/src/stores/useDroneStore.ts	Live-Log Rolling Buffer auf 1000 Zeilen.
webui/src/services/CameraMediaService.ts	0x39/0x3A, Capture readiness, Device-busy Retry, 0x20-Accumulator, 0x1B-Framepuffer/Flags, Disconnect/Resync.
webui/src/services/UsbTransportService.ts	Disconnect-Reset und Kamera-Statussync nach Reconnect.
README.md	Live-Log und Kamera-/Medien-Zuverlässigkeit

	v1.4 dokumentiert.
PROTOCOL.md	Command-Response vs. Notification/CameraLog sowie 0x20/0x1B-State-Regeln präzisiert.

4. Prüfungen

Prüfung	Ergebnis	Bemerkung
TypeScript/Vue: vue-tsc --noEmit	BESTANDEN	Keine Typfehler im geänderten WebUI-Quellstand.
Vite Produktionsbuild	UMGEBUNGSBEDINGT OFFEN	Geliefertes node_modules enthält Windows-Rollup-Binaries; @rollup/rollup-linux-x64-gnu fehlt.
Android/Kotlin Gradle Compile	UMGEBUNGSBEDINGT OFFEN	Gradle 8.11.1 müsste heruntergeladen werden; Netzwerkzugriff in der Ausführungsumgebung nicht verfügbar.
Reale ATOM-Geräteabnahme	OFFEN	K1-K8 benötigen reale Kamera-/Passthrough-Antworten.

5. Abnahmekriterien aus dem Änderungsauftrag

5.1 Live-Log L1-L6

ID	Umsetzungsstand	Prüfstatus
L1	Header liegt außerhalb des scrollenden Inhalts; Zähler/Save/Clear bleiben sichtbar.	Code umgesetzt; Browser-Sichtprüfung empfohlen.
L2	addLog() begrenzt deterministisch auf 1000 und verwirft älteste Einträge.	Code umgesetzt; TypeScript geprüft.
L3	Save Log exportiert store.logs-Snapshot, maximal 1000 Einträge.	Code umgesetzt; Browser-Downloadtest empfohlen.
L4	Clear setzt logs auf leeres Array; Zähler ist reaktiv.	Code umgesetzt.
L5	Neue Meldungen nach Clear werden normal angefügt.	Code umgesetzt.
L6	Export liest Snapshot und verändert Live-Logging nicht.	Code umgesetzt; Browser-Laufzeittest empfohlen.

5.2 Kamera/Medien K1-K8

ID	Umsetzung	Geräteabnahme
K1	0x39 wird vor Statusbyte-Auswertung als CameraLogData abgefangen.	OFFEN

K2	0x03 ACK -> 0x3A und/oder bestätigendes 0x02 -> erst dann Capture.	OFFEN
K3	Device busy: Timer stop, 350-ms-Backoff, 0x02-Resync, maximal ein Retry.	OFFEN
K4	0x20-JSON wird fragmentweise gepuffert und erst vollständig geparkt.	OFFEN
K5	flag=0 schreibt/ wartet; kein neuer 0x1B bis flag=1.	OFFEN
K6	0x1B-Body wird anhand deklarierter Länge bis zur Vollständigkeit gepuffert.	OFFEN
K7	Disconnect beendet Pending Capture und erzwingt 0x02-Resync nach Reconnect.	OFFEN
K8	Disconnect beendet Download sichtbar; kein stilles Resume mit altem Offset.	OFFEN

6. Grenzen und Geräteprüfung

Die Codeänderungen setzen die im Änderungsauftrag beschriebenen Zustands- und Pufferregeln um. Ohne angeschlossene Drohne können weder die konkrete 0x39-Quellkennung einzelner Firmwarevarianten noch die zeitliche Reihenfolge von 0x03/0x3A/0x02, fragmentierten 0x20-Antworten und mehrteiligen 0x1B-Blöcken real verifiziert werden. Deshalb bleiben K1-K8 als reale Abnahme offen und dürfen nicht als am Gerät bestanden ausgewiesen werden.

7. Empfohlene Geräteabnahme

- Direkt nach Neustart ohne Kamera-Button prüfen, dass 0x39 nur als [CAMERA/...] Log erscheint und keine Command-failed-Warnung erzeugt.
- Foto aus VIDEO-Modus und Video aus PHOTO-Modus testen; Logreihenfolge 0x03 -> Bereitschaft 0x3A/0x02 -> Capture prüfen.
- Device-busy provozieren und kontrollieren, dass exakt ein Retry erfolgt und kein zusätzlicher alter Timeout nachläuft.
- Galerie mit fragmentierter 0x20-Antwort laden; kein „unterminated string“, Metadaten am Ende vollständig.
- Große JPG/MP4 herunterladen; flag=0/1/2 und Offsetfortschritt im Live-Log prüfen; Datei auf Android gegen Größe/Öffnbarkeit verifizieren.
- Während Capture und Download WebSocket/Passthrough trennen; sichtbarer Fehler, danach Reconnect + 0x02-Statussync und keine alten Pending-States.

8. Änderungshistorie

Version	Datum	Status	Änderung
v1.3	2026-10-02	Vorgänger	Capture-State-Machine mit Modussynchronisation.
v1.4	2026-10-02	Software umgesetzt / Geräteabnahme offen	Live-Log v1.1 sowie 0x39/0x3A, 0x20-Fragmentierung, 0x1B-Teilframes und Passthrough-Reconnect umgesetzt und dokumentiert.